RAGAS, LLM-as-Judge & Guardrails

Bản tóm tắt tự học cho học viên

> **Mục tiêu của tài liệu**
>
> Giúp học viên nối các khái niệm trong slide thành một hệ thống: đo chất lượng AI bằng eval, tìm nguyên nhân lỗi, rồi đặt guardrails đúng vị trí để ngăn lỗi đến người dùng. Nội dung giữ đúng framing của bài giảng, thêm ví dụ và mental model để dễ tự học.

Cách dùng nhanh

- **Nếu cần ôn thi:** đọc các bảng "Cần nhớ" và trang Cheat Sheet cuối.
- **Nếu đang build RAG/Agent:** đọc theo luồng Evaluation -> RAGAS -> Judge -> Hallucination -> Guardrails -> Production.
- **Nếu debug hệ thống:** bắt đầu từ score/failure pattern, không thay model hoặc prompt một cách cảm tính.

Nguồn: tổng hợp từ slide "RAGAS, LLM-as-Judge & Guardrails - AICB-P2T3, Ngày 24" (74 trang).

1. Bức tranh lớn: Eval phát hiện, Guardrails ngăn chặn

Thông điệp xuyên suốt của bài: một demo chạy được chưa đủ để đưa AI vào production. Eval trả lời "hệ thống đang sai ở đâu và mức độ ra sao"; guardrails trả lời "làm sao để lỗi nguy hiểm không tới user". Hai phần chỉ có ý nghĩa khi team định nghĩa rõ thế nào là một output tốt.

```
User / Test case
      |
      v
RAG / Agent  --->  EVAL: đo chất lượng, drift, failure pattern
      |
      v
GUARDRAILS: chặn / sửa / cảnh báo trước khi output tới user
      |
      v
Production monitoring -> failure -> regression test -> CI/CD gate
```

> **Mental model quan trọng**
>
> Không có một metric duy nhất đại diện cho "AI tốt". Cần nhiều tầng eval: rẻ và rộng ở dưới, đắt và sâu ở trên; đồng thời cần defense-in-depth ở guardrails vì một lớp luôn có false negative.

Evaluation nên được thiết kế theo 4 góc nhìn

Góc nhìn	Câu hỏi chính	Ví dụ trong bài
Reference-based	Có ground truth; answer đúng đến đâu?	Exact match, Answer Correctness, Context Recall
Reference-free	Không có ground truth; answer có bám context / đúng trọng tâm?	Faithfulness, Answer Relevancy
Offline	Trước deploy / mỗi PR có regression không?	Golden set, RAGAS, pytest/DeepEval
Online	Sau deploy có drift / failure mới không?	Sample 1-5% production traffic, async eval

Tham chiếu slide PDF: trang 7-14, 74

2. Evaluation Foundations - đo đúng chỗ trước khi tối ưu

2.1 Eval pyramid L1 -> L4

Level	Loại eval	Coverage / vai trò	Điểm mạnh
L1	Heuristic: regex, schema	100% request	Rẻ, realtime, bắt lỗi cấu trúc
L2	Component eval: RAGAS	10-20% sample	Bắt lỗi retrieval/generation theo module

Level	Loại eval	Coverage / vai trò	Điểm mạnh
L3	LLM-as-Judge	1-5% sample	Đánh giá holistic, semantic
L4	Human eval	~0.1% sample	Gold standard, dùng để calibrate

Quy tắc thiết kế: tầng dưới phải rộng và rẻ; tầng trên hẹp nhưng sâu. Nếu đảo ngược - ví dụ 100% human eval - chi phí sẽ không scale.

2.2 Component eval và end-to-end eval phải đi cùng nhau

- **Component eval:** cho biết module nào hỏng. Retrieval có thể đo Context Precision/Recall; Generation đo Faithfulness/Answer Relevancy.
- **End-to-end eval:** đo trải nghiệm final answer. Một answer có thể "đúng" nhưng off-topic hoặc quá tốn chi phí.
- **Agent eval:** không chỉ nhìn final answer; phải đo trajectory correctness, tool selection, step efficiency, cost/task và final quality.

> **Ví dụ Agent**
>
> Agent trả đúng "Doanh thu Q3 FPT" nhưng gọi Google Search 5 lần thay vì internal_finance_db. Final answer đúng nhưng trajectory sai: tốn chi phí, có thể lộ query ra ngoài. Đây là lý do agent quality != final answer quality.

2.3 Bias-variance trong eval

Vấn đề	Biểu hiện	Cách xử lý theo slide
High bias	Test set không cover edge case; score đẹp nhưng không giống production	Mở rộng test set, đưa production failures vào regression suite
High variance	Score dao động giữa runs; judge không deterministic	Tăng n, temp=0, swap-and-average

Tham chiếu slide PDF: trang 9-14

3. RAGAS Deep Dive - 4 metric, 4 failure mode khác nhau

Metric	Đo quan hệ	Bắt lỗi gì?	Cần ground truth?
Faithfulness	Answer <-> Context	Hallucination / claim không được context support	Không
Answer Relevancy	Answer <-> Question	Off-topic / không trả đúng điều user hỏi	Không
Context Precision	Retrieved chunks + ranking	Chunk liên quan bị xếp thấp / ranking kém	Không theo framing slide
Context Recall	Context <-> Ground truth	Thiếu thông tin cần để trả lời đầy đủ	Có

3.1 Faithfulness - answer có được context hỗ trợ?

Cơ chế trong slide: (1) LLM tách answer thành factual claims; (2) với từng claim, kiểm tra entailment từ retrieved context; (3) Faithfulness = số claim được support / tổng số claim.

> **Ví dụ**
>
> Answer có 3 claim: doanh thu 50 nghìn tỷ, FPT lớn nhất VN, 70k nhân viên. Context chỉ support 2 claim sau -> Faithfulness = 2/3 = 0.67. Ý nghĩa: answer nghe hợp lý nhưng có một phần không grounded.

3.2 Answer Relevancy - không phải cosine trực tiếp Q với A

Slide dùng reverse-question generation: từ answer, LLM sinh ra khoảng 3 câu hỏi mà answer đó có thể trả lời; sau đó đo similarity giữa original question và các reverse questions. Nếu answer nói nhiều về FPT nhưng không trả lời "bao nhiêu nhân viên", similarity Q-original với reverse-Q sẽ thấp -> AR thấp.

3.3 Context Precision và Context Recall

- **Context Precision:** quan tâm thứ tự relevance; relevant chunk ở rank thấp có thể không vào prompt vì context window giới hạn. CP thấp -> xem lại retriever, re-ranker, hybrid search.
- **Context Recall:** tách ground truth thành các câu/ý, kiểm tra ý nào được retrieved context support. CR thấp -> chunks thiếu hoặc indexing chưa đầy đủ; có thể mở rộng top-k hoặc sửa ingestion/index.

3.4 Đọc score như một pattern, không như một con số cô lập

Pattern	Chẩn đoán nhanh	Hành động gợi ý từ slide
F thấp	Generation bịa / không bám context	Tighten prompt, thêm NLI/faithfulness guardrail
AR thấp	Answer off-topic	Sửa prompt instruction về relevance
CP thấp, CR cao	Lấy đủ thông tin nhưng rank sai	Re-ranker / hybrid search
CR thấp	Retriever thiếu thông tin	Improve indexing, expand top-k

3.5 Test set và pitfalls

Synthetic test generation trong slide chia Simple / Reasoning / Multi-context. Tỷ lệ mặc định 50/25/25 chỉ là điểm bắt đầu; nên tune theo traffic thật và manual-review một phần test trước khi dùng. Các pitfall cần nhớ: judge-model dependency, score drift khi đổi RAGAS version, test set stale và single-number obsession.

> **Targets trong bài giảng**
>
> General RAG: Faithfulness >= 0.85, Answer Relevancy >= 0.80, Context Precision >= 0.70, Context Recall >= 0.75. Slide nhấn mạnh threshold phải theo risk profile; medical/legal cần faithfulness cao hơn, creative task có thể relax.

Tham chiếu slide PDF: trang 15-25

4. LLM-as-Judge - scale evaluation nhưng phải kiểm soát bias

4.1 Pairwise vs Absolute scoring

Kiểu	Cách dùng	Ưu	Nhược
Absolute	Chấm 1 answer theo rubric 1-5	Theo dõi trend cross-run	Subjective, drift theo thời gian
Pairwise	So A vs B, chọn winner/tie	Ổn định hơn cho regression/A-B	Cần baseline, không có "điểm tuyệt đối"

4.2 Bốn bias phải nhớ

Bias	Biểu hiện	Mitigation chính
Position bias	Judge thích A/B theo vị trí dù quality ngang nhau	Swap-and-average; random order; cho phép tie
Length bias	Câu dài dễ được chấm cao hơn	Length-control; rubric ưu tiên concise; tách concise/comprehensive
Self-enhancement	Judge thiên về output cùng model/family/style	Cross-judge protocol, dùng judge khác family
Style/verbosity	Markdown/bullets trông "chuyên nghiệp" nên được điểm cao	Strip formatting; rubric content-only; multi-judge

4.3 Calibration với human - Cohen kappa

Kappa đo agreement giữa judge và human sau khi loại phần agreement do chance. Slide xem kappa >= 0.60 là production minimum; cần ít nhất khoảng 50 cặp human-judge, lý tưởng 200+ để confidence interval chặt hơn.

```
kappa = (P_observed - P_chance) / (1 - P_chance)
kappa < 0.40  -> judge yếu / bias mạnh
0.40-0.60     -> có thể dùng monitoring thận trọng
0.60-0.80     -> substantial, production minimum theo slide
```

4.4 Tier hóa judge để giảm cost

Architecture trong slide: T1 heuristic chạy 100%; T2 small LLM chạy khoảng 10%; T3 frontier judge chạy khoảng 1% hoặc chỉ khi borderline. Quy tắc escalation giúp giảm chi phí mà vẫn giữ một tầng deep review cho case khó.

> **Cần nhớ**
>
> LLM-as-Judge không thay human 100%. Human labels vẫn là anchor để calibrate. Nếu đổi judge model/version, phải re-baseline vì score có thể drift.

Tham chiếu slide PDF: trang 27-35

5. Hallucination Detection - dùng nhiều góc kiểm tra

Loại	Định nghĩa	Ví dụ	Detector trong slide
Intrinsic	Output mâu thuẫn context	Context 70k nhân viên, answer 50k	NLI entailment
Extrinsic	Output thêm fact không có trong context	Context chỉ nói FPT; answer thêm founder/năm thành lập	External KB / fact-checking

5.1 SelfCheckGPT - consistency-based

Không cần ground truth. Lấy answer gốc ở temp=0, sample thêm n câu ở temp>0, rồi đo consistency từng sentence bằng BERTScore/NLI. Intuition: phần factual thường ổn định giữa samples; phần hallucinated dễ mâu thuẫn. Trade-off là cost khoảng 6x nếu dùng 1+5 samples và latency tăng.

5.2 NLI - detector realtime cho RAG

Premise = retrieved context; hypothesis = từng sentence trong answer. Entailment -> factual; contradiction -> hallucination; neutral -> uncertain. Slide gợi ý dùng entailment score để flag/block theo risk profile.

5.3 Semantic Entropy

Sample nhiều answers, cluster theo semantic equivalence rồi tính entropy trên các cluster. Điểm hay là tách "khác cách diễn đạt" khỏi "khác fact". High semantic entropy -> model không ổn định về ý nghĩa -> uncertainty/hallucination risk cao; phù hợp offline monitoring hơn realtime vì tốn n lần generation.

> **Không có detector hoàn hảo**
>
> Trong production, nên combine: Faithfulness/NLI cho RAG realtime + SelfCheck/Semantic Entropy cho sampling/offline + external fact-check với extrinsic facts quan trọng. Mục tiêu là nhiều signal độc lập, không phải một score duy nhất.

Tham chiếu slide PDF: trang 36-41

6. Guardrails Foundations - 4 trục và defense-in-depth

Trục	Câu hỏi bảo vệ	Ví dụ / tools trong slide
Topical	Bot chỉ trả lời đúng phạm vi?	ValidTopic, NeMo Dialog Rails
Safety	Có nội dung độc hại?	Llama Guard 3, Moderation
Security	Có bị manipulate / injection?	Prompt Guard, Lakera Guard, Rebuff
Compliance	Có leak PII / vi phạm yêu cầu dữ liệu?	Presidio, Private AI, Skyflow

6.1 Defense-in-depth 4 layer

Layer	Vai trò	Ví dụ trong slide
L1 Input	Sanitize / classify trước khi vào LLM	PII redaction, Prompt Guard, topic scope
L2 LLM	Ràng buộc role / policy trong prompt	Structured system prompt
L3 Output	Kiểm tra answer trước khi trả user	Llama Guard, NLI hallucination check
L4 Audit	Log + sample async, không block request	Audit log, sample 1%

Slide đặt latency budget user-facing cho L1 + L3 dưới khoảng 100ms P95 và khuyến nghị chạy các validator độc lập song song thay vì sequential. L4 là async nên không nằm trong latency path.

6.2 Input validator chain - order matters

```
User input
   -> PII redact
   -> Injection check
   -> Topic scope
   -> LLM
```

- **Fail-fast:** validator reject thì có thể skip bước sau và trả lỗi/refusal sớm.
- **Fail-open vs fail-closed:** timeout validator thì allow hay block tùy risk profile.
- **VN PII:** slide kết hợp custom regex (CCCD/phone/tax code) + Presidio NER.

6.3 Output guardrails và over-filtering

Output có thể đi qua safety classifier (Llama Guard 3) và hallucination NLI trước khi return. Nhưng guardrail quá aggressive gây false positive, refuse rate cao, user học cách bypass và UX xấu. Slide gợi ý đo refuse rate, A/B threshold, có graceful fallback và review false positives định kỳ.

> **Nguyên tắc UX**
>
> Guardrail tốt nhất gần như "vô hình" với legitimate user: chặn đúng case nguy hiểm nhưng không làm người dùng bình thường cảm thấy bot từ chối mọi thứ.

Tham chiếu slide PDF: trang 42-48, 55-59

7. Prompt Injection, Session Poisoning & OWASP threat model

7.1 Direct vs Indirect Injection

Loại	Nguồn attack	Tại sao nguy hiểm	Defense chính
Direct	User input: "ignore previous...", jailbreak, DAN	Nhìn thấy được nhưng có thể bypass prompt rule	Input filter / Prompt Guard / explicit role rules
Indirect	RAG docs, web, tool results chứa malicious instruction	User không thấy; agent có thể silently obey	Separate user vs retrieved context, sandbox tools, retrieval rails

7.2 Năm attack patterns trong bài

- **DAN:** ép model giả vờ không có restriction.
- **Role-playing:** đổi persona để lách safety rule.
- **Payload splitting:** chia harmful intent thành nhiều phần vô hại riêng lẻ.
- **Encoding bypass:** Base64/ROT13/Unicode tricks; defense là decode rồi re-check.
- **Indirect injection:** malicious content nằm trong tài liệu/tool result chứ không trong user prompt.

7.3 Session Poisoning - lỗi nằm trong history

> **Attack flow**
>
> Turn 2 chứa malicious input và output bị block. Nếu hệ thống chỉ chặn output nhưng vẫn lưu malicious input vào conversation history, Turn 3 "Continue earlier" có thể khiến agent obey instruction độc hại đã nằm trong history.

Defense đúng theo slide: khi detect unsafe input, thay nội dung đã lưu trong history bằng placeholder như "[Message removed by safety filter]". Đây là ví dụ "architecture > tool": output classifier đơn lẻ không sửa được poisoned memory.

7.4 OWASP LLM Top 10 (2025) - tên threat cần nhận diện

ID	Threat	Mitigation trong slide
LLM01	Prompt Injection	Defense-in-depth, input filters
LLM02	Sensitive Info Disclosure	PII redaction, output filters
LLM03	Supply Chain	Pin model versions, vendor audit
LLM04	Data & Model Poisoning	Provenance check, RAG validation
LLM05	Improper Output Handling	Output validation, sandboxing
LLM06	Excessive Agency	Tool permissions, HITL
LLM07	System Prompt Leakage	Không để secret trong prompt
LLM08	Vector & Embedding Weaknesses	Embedding sanitization
LLM09	Misinformation	Faithfulness check, citation
LLM10	Unbounded Consumption	Rate limit, token cap

Tham chiếu slide PDF: trang 49-54

8. Production Patterns - eval và guardrail là continuous discipline

8.1 CI/CD eval gate

Gate	Bài test	Pass criteria theo slide
L1	Smoke 10 queries	Format/schema OK
L2	RAGAS 100-query golden set	F >= 0.85, AR >= 0.80; slide cũng dùng CP/CR ở eval suite
L3	Pairwise judge vs production	Win rate >= 50%
Security	30 adversarial inputs	Detection >= 95%

Pattern: mọi PR chạy eval trước merge; bất kỳ gate nào fail thì block merge, override cần manual approval + justification log. Đây biến eval từ hoạt động "khi rảnh" thành regression protection thực sự.

8.2 Production failure -> regression test loop

```
Production incident / complaint
          -> reproduce
          -> fix
          -> add expected-behavior test case
          -> future PRs run regression
```

Test set tốt không đứng yên. Slide khuyến nghị tag test với incident ID, duy trì failure taxonomy và review theo quý để biết pattern nào lặp lại.

8.3 A/B testing + Continuous Evaluation

- **A/B:** deploy B cho một phần traffic, evaluate A và B trên same query, dùng pairwise judge + statistical test; xem subgroup chứ không chỉ aggregate.
- **Continuous eval:** sample khoảng 1-5% production traffic, chạy async, aggregate theo giờ/ngày/feature/user segment, alert khi metric drift.
- **Drift sources:** prompt drift, model drift, data drift, user drift.

8.4 Compliance & case studies trong slide

Slide nối guardrails với compliance: cần audit log, human override cho high-risk decisions, kiểm soát cross-border personal data và data residency tùy regulation. Các case Air Canada, Samsung, DPD và Bing/Sydney minh họa bốn failure class: hallucination liability, PII/IP leak, behavior degeneration, prompt injection/persona drift.

> **Bài học từ case study**
>
> Failure production không nhất thiết vì model "dở". Nhiều lỗi xảy ra vì hệ thống thiếu eval gate, input/output controls, audit và defense-in-depth.

Tham chiếu slide PDF: trang 60-66

9. Blueprint để học viên tự triển khai

9.1 Lab 24 - 3 phase / 90 phút

Phase	Mục tiêu	Output nên có
A - RAGAS	50-question test set; 4 metrics; bottom-10 + failure clusters	Bảng score + chẩn đoán retrieval/generation
B - Judge	Pairwise, swap-and-average, Cohen kappa vs human, document bias	Judge pipeline đã calibrate
C - Guard	PII + topic, 20 adversarial, Llama Guard output, P95 latency	Guardrail stack + 1-page blueprint

9.2 Khi debug - đọc triệu chứng trước

Triệu chứng	Suy luận đầu tiên
Tất cả RAGAS < 0.5	Check judge/API/context format trước khi kết luận model dở
Faithfulness cao, AR thấp	Grounded nhưng off-topic -> prompt relevance
CP thấp, CR cao	Có đủ chunk nhưng ranking sai -> re-ranker
Cohen kappa < 0.4	Judge bias mạnh -> swap/cross-judge
Presidio miss PII VN	Thêm regex VN / model ngôn ngữ phù hợp
Guardrail quá restrictive	Tune threshold/category; đo false positive / refuse rate

9.3 Câu hỏi tự kiểm tra sau bài

- Vì sao Faithfulness cao không đảm bảo Answer Relevancy cao?
- Nếu CP thấp nhưng CR cao, bạn sửa retriever theo hướng nào?
- Tại sao pairwise judge thường phù hợp regression hơn absolute scoring?
- Bốn bias của LLM-as-Judge là gì và bias nào cần swap-and-average?
- Vì sao output blocking không đủ để chống Session Poisoning?
- Khi nào nên dùng fail-open và khi nào fail-closed cho validator timeout?
- Làm sao biến một production incident thành regression test trong CI/CD?

Tham chiếu slide PDF: trang 68-70

10. Cheat Sheet - 1 trang để ôn nhanh

Nếu bạn thấy...	Hãy nghĩ tới...	Việc làm đầu tiên
Answer bịa fact so với context	Faithfulness / intrinsic hallucination	Claim extraction + NLI entailment
Answer đúng context nhưng không trả Q	Answer Relevancy	Sửa instruction/relevance
Relevant chunk bị xếp thấp	Context Precision	Re-ranker / hybrid search
Thiếu facts để trả ground truth	Context Recall	Indexing / top-k / corpus coverage
Judge chọn câu dài/markdown	Length / style bias	Length-control, strip format, rubric
Judge thiên model của chính nó	Self-enhancement bias	Cross-judge
Unsafe input vẫn nằm trong memory	Session Poisoning	Sanitize/replace history at input layer
Bot refuse quá nhiều	Over-filtering	Tune threshold, measure refuse rate, fallback
Prod metric giảm theo thời gian	Drift	Async continuous eval + segment analysis

> **Ba câu chốt của Day 24**
>
> 1) Eval không phải optional: RAGAS + LLM-as-Judge là baseline. 2) Guardrails phải defense-in-depth: input / LLM / output / audit. 3) Judge có bias: phải mitigation + human calibration, không được tin một score tuyệt đối.

Tài liệu để tự học tiếp

- RAGAS Documentation - synthetic test generation và 4 core metrics.
- Zheng et al. 2023 - LLM-as-a-Judge with MT-Bench and Chatbot Arena.
- Chen et al. 2024 - judgement bias của human/LLM judge.
- Manakul et al. 2023 - SelfCheckGPT.
- Farquhar et al. 2024 - semantic entropy for hallucination detection.
- OWASP LLM Top 10 (2025).
- Microsoft Presidio; Meta Llama Guard 3; Google ADK Model Guardrails cookbook.

Gợi ý học tiếp từ slide: Reliability & Production-Ready Agent - circuit breakers, fallback chains, semantic caching và multi-provider routing.

Tham chiếu slide PDF: trang 71-74

> **Một câu để nhớ cả bài**
>
> Eval cho biết vấn đề gì; guardrails ngăn vấn đề tới user. Cả hai đều bắt đầu bằng việc định nghĩa rõ "tốt" là gì.